

YavOracle: A Typed Rust Framework for Interactive Proofs, Coverage-Safe Fiat–Shamir, and Fischlin Transforms

Yavor Litchev*

Mentor: Dan Boneh†

Abstract

We present YavOracle, a Rust framework for programming interactive proof systems that enforces coverage safety for Fiat–Shamir and Fischlin transforms by construction. Coverage safety is the requirement that every prover-controlled, verification-relevant value that can influence a verifier’s decision is bound into the transcript before any challenge or final decision derived from that transcript. While classical soundness theorems for Fiat–Shamir and Fischlin assume this property implicitly, recent real-world vulnerabilities have shown that hand-written transcript code frequently violates it through omitted hash inputs.

YavOracle addresses this gap with three layers. At the message level, a lightweight DSL based on `#[derive(FsrBindable)]` annotates fields as obligatory and derives canonical binding code plus per-message obligation masks. At the transcript level, a typestate `Transcript<PENDING, 0>` tracks outstanding obligations as a const-generic bitmask; absorption operations discharge obligations, and `challenge()` is only available when the pending mask is zero. At the proof level, `prove!/verify!` macros parameterized by `transform = "fs" | "fischlin"` and a `require` list of labels statically check that all required bindings occur, independent of the chosen transform.

We implement YavOracle and evaluate it on Schnorr, Σ -OR, and Σ -AND protocols under interactive, Fiat–Shamir, and Fischlin runtimes. Our case studies show that representative omission bugs are rejected at compile time with precise diagnostics, while the correct protocols incur negligible overhead and can switch transforms without code changes.

1 Introduction

Non-interactive zero-knowledge arguments are now part of the critical path for real systems: blockchains, rollups, light clients, and privacy-preserving tokens all deploy protocols that ultimately rely on converting interactive proofs into non-interactive ones, usually via the Fiat–Shamir (FS) transform in the Random Oracle Model (ROM) [6] [16]. In this transformation, the verifier’s random challenges are replaced by hashes of the public statement and the transcript so far; security proofs typically assume that every prover-controlled value that influences verification is correctly hashed into the oracle state.

Implementors, however, do not work in the ROM—they write code. In practice, FS-style code is written by hand, often as a series of ad hoc hash updates sprinkled throughout prover and verifier implementations. When protocols evolve, or when multiple sub-protocols are combined, it is easy to forget to bind some prover-chosen value into the transcript,

to reuse labels inconsistently, or to hash values in the wrong order. These coverage bugs are not theoretical: recent incidents in production systems have traced security vulnerabilities back to “missing math” in zero-knowledge components, where invalid proofs could be accepted because a necessary constraint or binding was omitted [8] [10] [15] [7]. Such bugs are hard to detect with tests alone because almost all honest executions look correct.

Existing libraries help structure transcript programming but do not fully address this problem. For example, sponge-based Fiat–Shamir libraries such as `spongefish` expose a high-level duplex-sponge interface that minimizes permutation calls and provides good domain-separation tools, and they can be plugged into a range of cryptographic frameworks [12] [5]. However, these tools emphasize shape (the sequence of absorb/squeeze operations) rather than coverage: they do not track which fields are prover-controlled and verification-relevant, and they do not

*Stanford University. Email: ylichev@stanford.edu

†Stanford University. Email: dabo@cs.stanford.edu

offer static guarantees that every such field has been bound before challenges are sampled. A protocol that forgets to absorb a critical commitment will still compile and run; security depends on the programmer’s discipline.

This paper proposes coverage safety as an explicit design goal for Fiat–Shamir and Fischlin-style implementations, and presents YavOracle, a Rust framework that enforces this property by construction. Our central idea is to treat “prover-controlled, verification-relevant” data as an effect on the transcript type. Protocol authors annotate message fields to indicate which ones must be bound before the next challenge; these annotations are compiled into a bit-mask of obligations. A transcript `Transcript<PENDING, 0>` carries this obligation set as a const-generic parameter. Absorbing a message clears its corresponding bits, and sampling a challenge is only type-correct when the pending set is empty. If a relevant value is not absorbed, the call to `challenge()` simply does not type-check, turning coverage bugs into compile-time errors rather than subtle soundness flaws [20].

On top of this core calculus, YavOracle provides a small proof DSL in the `fsr-proof-dsl` crate. The `prove!` and `verify!` macros accept a parameter `transform = "fs" | "fischlin"`, a list of required labels `require = ["label1", "label2", ...]` representing coverage obligations, and a `bind` closure that specifies how to absorb message bytes under each label. The macros statically analyze this closure to check that all required labels are actually absorbed; if any are missing, the compilation fails with a targeted diagnostic suggesting the missing absorb call. Crucially, this coverage check is transform-agnostic: the same `require` and `bind` specification is used whether the underlying runtime is a simple FS hash oracle or a Fischlin oracle with online extraction. Thus, protocol authors can switch `transform = "fs"` to `transform = "fischlin"` without touching protocol code or weakening coverage guarantees.

YavOracle also supports an interactive runtime, where the prover and verifier communicate over a concrete channel and the verifier samples challenges directly. All three modes—interactive, FS, and Fischlin—are exposed through a common oracle abstraction, so that the same typed protocol code can be executed in different environments by swapping only the oracle type. At the lowest level, the `fsr-core` crate provides the typed transcript, oracle traits, and Fischlin parameter checks (including support for n-special soundness with a configurable bit-budget). The `fsr-bind-derive` crate offers `#[derive(FsrBindable)]`, which automatically generates canonical binding logic and obligation masks from anno-

tated message structs. The `fsr-proof-dsl` crate then layers the `prove! / verify! / proof!` macros and coverage lints on top of this foundation.

We evaluate YavOracle on canonical Σ -protocols (Schnorr, Σ -OR, Σ -AND) and multi-round sub-protocols, providing both “correct” and “buggy” implementations. The correct versions compile and run under interactive, FS, and Fischlin modes; the buggy versions, which omit bindings or misplace them after challenges, are rejected at compile time by the type-state system or by the DSL coverage lints. We also reconstruct a representative omission bug in a simplified setting inspired by real-world failures, and show that YavOracle’s coverage checks prevent such code from compiling.

To summarize, this paper makes the following contributions:

1. **Coverage safety as a first-class property:** We articulate coverage safety—the requirement that every prover-controlled, verification-relevant value must be bound into the transcript before challenges or final decisions—as a concrete security objective for FS/Fischlin implementations, and we formalize it in a simple transcript calculus.
2. **A typed transcript framework with effect-like obligations:** We design and implement `Transcript<PENDING, 0>` in Rust, where `PENDING` is a type-level set of obligations derived from annotated messages. This typestate discipline uses const generics to gate `challenge()` on the absence of outstanding obligations, turning omission bugs into type errors.
3. **Transform-agnostic FS/Fischlin DSL:** We introduce a proof DSL (`prove!`, `verify!`, `proof!`) whose coverage checks are independent of the underlying transform; the same specification can instantiate FS or Fischlin mode via a `transform` parameter while retaining identical coverage guarantees.
4. **A Rust implementation and case studies:** We implement YavOracle as a set of Rust crates and apply it to a suite of Σ -protocols and composed sub-protocols, demonstrating compile-time rejection of curated omission bugs and low overhead compared to hand-rolled transcripts.

The remainder of the paper is organized as follows: Section 2 reviews interactive proofs, Σ -protocols, the classical Fiat–Shamir transform, and Fischlin’s transform with n-special soundness. Section 3 defines coverage safety and illustrates omission bugs through real-world case studies. Section

4 presents a high-level design overview of YavOracle, and Section 5 formalizes the transcript calculus and coverage property. Section 6 describes the Rust implementation, Section 7 evaluates correctness and performance on a corpus of examples, Section 8 discusses related work, and Sections 9 and 10 conclude with limitations and future directions.

2 Background

This section recalls the cryptographic concepts underlying our framework: public-coin interactive proofs and Σ -protocols, the Fiat–Shamir transform in the Random Oracle Model, and Fischlin’s transform with n -special soundness. Our goal is not to re-prove known theorems, but to fix terminology and highlight the aspects where implementation details are critical.

2.1 Interactive Proofs and Σ -Protocols

An interactive proof system for a language (L) is a two-party protocol between a prover (P) and a verifier (V) where, on input (x), the prover tries to convince the verifier that ($x \in L$). Soundness requires that no polynomial-time adversarial prover can make the verifier accept false statements (inputs ($x \notin L$)) with more than negligible probability, whereas completeness requires that an honest prover convinces the verifier on true statements. In this work we focus on public-coin interactive proofs, where the verifier’s messages are uniform random coins visible to the prover.

A Σ -protocol (Sigma protocol) is a three-move public-coin proof of knowledge for a relation (R), typically of the form “the prover knows a witness (w) such that ($R(x,w)$) holds.” The canonical template has the shape:

1. **Commitment:** Prover samples randomness (r), computes a commitment ($t = \text{Com}(x;r)$), and sends (t) to the verifier
2. **Challenge:** Verifier samples a random challenge (e) from some challenge space and sends it to the prover
3. **Response:** Prover computes a response ($z = \text{Resp}(x, w, r, e)$) and sends (z) back

[16] [22]

The verifier then runs a deterministic check ($\text{Verify}(x, t, e, z)$) and accepts or rejects accordingly. Σ -protocols usually satisfy:

Special soundness: Given two accepting transcripts (t, e, z) and (t, e', z') with the same commitment t but distinct challenges $e \neq e'$, one can efficiently extract a witness w . Intuitively, if a prover can answer two different challenges for the same commitment, it must “know” the underlying secret.

Honest-verifier zero knowledge: There exists a simulator that, given only x and access to the verifier’s coin tosses, can produce transcripts indistinguishable from real interactions.

Many identification schemes and zero-knowledge proofs follow this pattern: Schnorr’s proof of knowledge of a discrete logarithm is perhaps the most famous example. In Schnorr, for a cyclic group G of prime order q with generator g , the statement is “I know (x) such that $y = g^x$ ”; the commitment is $t = g^r$, the challenge is $e \leftarrow Z_q$, the response is $z = r + ex \bmod q$, and verification checks $g^z = t \cdot y^e$. In this protocol, the commitment and response are prover-controlled values whose correct binding to the challenge is essential for soundness.

Σ -protocols admit various compositions (AND, OR, threshold) and multi-round generalizations, which are heavily used as sub-protocols inside more complex proof systems (e.g., in Groth–Sahai-style constructions or modern SNARKs). In all cases, the security proofs are formulated at the level of transcripts—sequences of commitments, challenges, and responses—rather than at the level of code. [16] [22]

2.2 The Fiat–Shamir Transform in the Random Oracle Model

The Fiat–Shamir heuristic (or transform) is a generic way to convert a public-coin interactive proof or Σ -protocol into a non-interactive proof by replacing the verifier’s random challenges with outputs of a hash function modeled as a random oracle [6] [16]. Given a Σ -protocol as above and a public statement (x), the non-interactive prover proceeds as follows:

1. Run the commitment step to obtain (t)
2. Compute the challenge as

$$e = H(x, t, \text{aux}),$$

where (H) is a hash function interpreted as a random oracle and (aux) may include additional context (protocol identifiers, labels, etc.)

3. Compute the response (z) as in the interactive protocol
4. Output the proof ($\pi = (t, z)$) (the challenge is recomputed by the verifier)

The verifier, given (x) and $(\pi = (t, z))$, recomputes $(e = H(x, t, \text{aux}))$ and checks $(\text{Verify}(x, t, e, z))$.

Security proofs for FS are typically carried out in the Random Oracle Model (ROM), where (H) is an idealized oracle that returns independent uniform values for each new query. Under appropriate conditions (e.g., special soundness and honest-verifier zero knowledge), one can show that if the interactive protocol is sound and zero-knowledge, then the FS-transformed scheme is a sound non-interactive proof of knowledge in the ROM.

From an implementation perspective, however, these proofs assume that the entire transcript relevant to soundness is correctly hashed:

- Every prover-controlled value that can influence acceptance (e.g., commitments, auxiliary inputs) must be included in the random oracle query that defines the challenge
- Domain separation must be respected: different protocol steps, roles, and sub-protocols should map to distinct inputs to prevent cross-protocol interactions
- The order of absorption matters: challenges should not depend on values that are only specified “later” in the transcript, and replaying the transcript must be deterministic

If any prover-controlled, verification-relevant value is omitted from the hash input—because, for instance, the code forgets to call `absorbed_commitment.update(hash_state)`, the theoretical security guarantees no longer apply. The scheme may remain perfectly sound in most test scenarios while being exploitable by an adversarial prover that exploits the missing dependency, as has been observed in real systems [15]. These are precisely the coverage bugs we aim to prevent.

Modern FS implementations often use sponge-based constructions (e.g., Keccak or Poseidon), where transcript programming consists of a sequence of sponge “absorb” and “squeeze” operations. Libraries like `spongefish` expose an abstract duplex sponge interface that can be instantiated with different permutations and codecs, providing efficient and flexible hash-based transcripts [12] [5]. However, while these libraries address performance and interoperability and offer tools for domain separation, they generally leave it to the programmer to ensure that all relevant prover-controlled values are absorbed before squeezing challenges. If a field is forgotten, the library cannot tell.

2.3 Fischlin’s Transform and n-Special Soundness

Fischlin’s transform provides an alternative to Fiat–Shamir for turning Σ -protocols into non-interactive proofs of knowledge, with the key feature that extraction is straight-line (no rewinding) and can be made efficient even in concurrent settings. At a high level, Fischlin considers an underlying Σ -protocol and defines a new non-interactive protocol where the prover internally searches for accepting transcripts that satisfy a hash-based predicate [17] [14].

Informally, for a given instance (x) , the Fischlin prover:

- Samples internal randomness and generates several candidate commitments and responses
- For each candidate transcript fragment, computes a hash value

$$h_i = H(x, t_i, e_i, z_i),$$

where (t_i, e_i, z_i) are commitment, challenge, and response elements, and (H) is modeled as a random oracle

- Accepts and outputs only those transcripts where h_i satisfies a predicate such as “the first b bits of h_i are zero,” effectively encoding a proof of knowledge into a rare hash event
- Packages the surviving transcripts (or compressed information derived from them) as the non-interactive proof

[17] [14]

The verifier recomputes the same hash values from the provided transcripts and checks both the underlying Σ -protocol verification and the hash predicate. Fischlin’s main result is that, under appropriate conditions, one can obtain communication-efficient non-interactive proofs of knowledge with online extractors in the ROM. Importantly, the transform provides straight-line extraction even in concurrent executions, which makes it attractive for protocol composition and UC-style reasoning.

A central parameter in Fischlin’s analysis is the bit budget used by the hash predicate. If the predicate demands that the first (b) bits of a hash output are zero, then each transcript has success probability 2^{-b} . By repeating the protocol ρ times and checking all repetitions, one can balance soundness, communication, and running time. For protocols with ordinary 2-special soundness, a typical condition is that

$$\rho \cdot b \geq \kappa_c,$$

where κ_c is the desired computational security parameter, up to small losses. For Σ -protocols with n -special soundness, where extraction may require up to (n) accepting transcripts with the same commitment, one effectively loses $\lceil \log_2(n-1) \rceil$ bits per repetition, yielding a condition of the form

$$\rho \cdot (b - \lceil \log_2(n-1) \rceil) \geq \kappa_c.$$

In practice, these constraints mean that implementations must carefully choose (ρ, b) given n_{special} and κ_c , and must ensure that the hash predicate is applied to the correct set of transcript values. Just as in the classical FS setting, omitting a prover-controlled value from the hash input—or mistakenly hashing values in the wrong phase—can invalidate the theoretical soundness guarantees. Moreover, because Fischlin’s transform internalizes a search over transcripts, it is easy for implementors to “sneak in” additional randomness or state that is not properly reflected in the hash predicate, especially when integrating with existing code or optimizing for performance [17] [14].

From a software-engineering standpoint, FS and Fischlin share the same implementation challenge: they are conceptually simple transforms defined at the level of ideal transcripts, but real systems must implement them as lines of code that manipulate hash states and message structures. Ensuring that every prover-controlled, verification-relevant value is bound into the transcript in the right place and under the right label is essential for soundness, yet existing libraries largely treat this as a discipline issue. The rest of this paper develops a framework in which this discipline is enforced by the type system and by a small DSL, uniformly across interactive, FS, and Fischlin runtimes, and parameterized by n_{special} and the relevant bit budgets.

3 Omission Bugs and Coverage Safety: Problem Statement

This section refines the problem we aim to solve. We first illustrate how omission bugs manifest in deployed systems, using recent incidents as case studies. We then formalize coverage safety as a desired property of Fiat–Shamir and Fischlin implementations, and conclude with the threat model and scope of our guarantees.

3.1 Omission Bugs in Practice

A recurring pattern in the failure of Fiat–Shamir (FS) and related constructions is not an incorrect theo-

rem or a broken primitive, but a missing line of code: some prover-controlled value that ought to be hashed into the transcript is silently omitted. This class of mistakes has been documented in multiple public incidents.

Regarding FS omissions in general, the maintainers of Merlin transcripts explicitly highlight that the FS heuristic is underspecified at the level of code, and that hash-chain optimizations and transcript libraries have repeatedly led to bugs where implementors “forgot to hash the final round” or omitted contextual data such as group descriptions [4] [3]. The 0xPARC zk-bug tracker collects a series of vulnerabilities in Plonk implementations where some public inputs were not included in the Fiat–Shamir hash; in each case, the missing inputs gave the prover extra degrees of freedom to forge proofs that satisfied the verifier’s checks despite violating the intended statement [7]. These examples already fit a common template: *A value chosen or influenced by the prover, which affects the verifier’s accept/reject decision, is not included in the FS transcript, whereas the protocol’s soundness proof assumes that it is.* A particularly concrete illustration of this pattern appears in Solana’s Token-2022 confidential transfer extension. In June 2025, the Solana Foundation disclosed a critical vulnerability in the on-chain ZK ElGamal Proof program, which verifies zero-knowledge proofs for confidential token amounts. The post-mortem states that *“In the on-chain ZK ElGamal Proof program, a component was not included in a hash used to generate a transcript for the Fiat–Shamir Transformation. A sophisticated attacker could use this unhashed component to develop a forged proof of an unauthorized action that passes verification.”* [18]

Subsequent community analyses catalogued two distinct Fiat–Shamir implementation bugs: (1) certain proof response elements were omitted from the transcript, and (2) a prover-controlled challenge $c_{\text{max_proof}}$ was not included in the prover’s hash input. [19]

Both bugs are archetypal coverage failures: values that appear in the protocol’s symbolic description and influence verification were simply not bound into the FS hash. As the Solana and third-party write-ups emphasize, an attacker who understands the protocol can exploit this slack to construct transcripts that satisfy the on-chain checks without corresponding to a valid confidential transfer, enabling unauthorized minting or withdrawals.

From the perspective of a transcript library such as spongefish, these bugs are invisible: the library correctly absorbs whatever bytes the caller passes. The problem is not in the sponge itself, but in the

choice of what to absorb. A typical FS helper might expose an API of the form

```
1 fs.absorb("commitment", &t);
2 let e = fs.challenge("e");
3 fs.absorb("response", &z);
```

If the programmer never calls `fs.absorb("response", &z)` for some response field, or hashes it under the wrong label or phase, the library has no inherent way to detect that this is incorrect. The resulting code compiles, unit tests with honest provers appear to pass, and yet the implementation violates the assumptions under which the protocol is proven sound.

Across these incidents, several common features emerge:

- The underlying protocol is specified as a sequence of messages, challenges, and checks where particular values (commitments, responses, or auxiliary public inputs) are assumed to be part of the random-oracle input
- The implementation represents these values as fields in data structures, but there is no mechanical linkage between “this field is verification-relevant” and “this field is included in every appropriate hash.”
- When the protocol evolves, e.g. a new component is added to the proof, or an OR-composition is introduced, developers may update the message structs and verification logic but fail to update every FS hash site accordingly
- The bug is thus a coverage failure: not all verification-relevant, prover-controlled state is covered by the FS binding

In this paper we use “omission bug” to refer specifically to this pattern. We do not consider, for example, missing constraints within an arithmetic circuit that never appear as transcript elements at all; our focus is on the layer where interactive protocols are transformed into FS or Fischlin-style non-interactive arguments.

3.2 Coverage Safety as a Design Objective

The above case studies motivate a clear design objective for FS/Fischlin implementations.

Let us write $\mathcal{T}$ for the set of transcript events in a protocol execution, and let $\mathcal{V}$ be the set of values that can influence the verifier’s accept/reject decision

in that execution. Among these values, we distinguish those that are prover-controlled: either chosen directly by the prover (e.g., commitments, responses) or derived from prover-side state and randomness.

Informally, we say that an implementation is coverage-safe if, for every execution and for every prover-controlled value $v \in \mathcal{V}$, the following holds:

- Either v is not supposed to appear in the FS or Fischlin hash inputs according to the protocol specification (e.g., it is an internal helper value that never affects verification directly), or else
- v is guaranteed to be bound into the transcript—i.e., it contributes to the input of the random oracle—strictly before any challenge or final decision that depends on v is derived

Binding here should be interpreted at the level of the actual hash function or sponge state used in the implementation: if the implementation realizes the FS transform via a duplex sponge, then coverage safety requires that the bytes representing v , together with appropriate labels and domain separation tags, have been absorbed into the sponge state before any call that samples the challenge or evaluates the Fischlin predicate.

This property has several important nuances:

Selective relevance: Not all fields of a message need to be bound; some may carry ephemeral hints or logging information that do not influence the verifier’s decision. Coverage safety therefore presumes a classification of fields as verification-relevant or not. In our framework this classification is explicit: developers mark verification-relevant fields with annotations, and all such fields must be bound.

Phase sensitivity: In multi-round protocols and transforms like Fischlin’s, binding must occur in the correct phase. A response that is supposed to depend on a challenge e must not be hashed into e itself. Conversely, a commitment that is intended to be fixed before seeing the challenge must be bound prior to sampling e . Coverage safety therefore encompasses both “whether” and “when” a value is bound.

Transform-agnosticism: The property should be defined relative to an abstract transcript semantics, not to a particular realization of FS or Fischlin. That is, if a protocol has a symbolic description in which certain values are bound before certain challenges, then any admissible realization—interactive, FS, or Fischlin—should respect the same coverage discipline.

Existing libraries provide some of the ingredients for coverage safety: transcript APIs discourage direct manipulation of hash states, and domain separation

labels help prevent accidental collisions between unrelated contexts [11]. However, they do not connect the set $\mathcal{V}$ of verification-relevant, prover-controlled values to the set of binding operations in the implementation. The link between “this field is critical to soundness” and “this field is absorbed before the corresponding challenge” remains informal.

YavOracle’s core technical goal is to make coverage safety syntactic: developers declare which fields are verification-relevant, and the type system and DSL ensure that there is no code path in which a challenge or decision is derived before all such fields are bound. At the transcript level, this is enforced through obligation bitmasks in types such as `Transcript<PENDING, O>`. At the proof-DSL level, the `prove!` and `verify!` macros take an explicit list of required labels and statically verify that the associated binding closure absorbs each of them before issuing challenges, regardless of whether the underlying runtime is FS or Fischlin.

3.3 Threat Model and Scope

We conclude this section by clarifying the threat model under which coverage safety is meaningful and the scope of what YavOracle aims to guarantee.

Adversary model: We assume an adversarial prover who may deviate arbitrarily from the prescribed protocol and who has full control over any prover-side code not enforced by our framework. The verifier and the transcript/oracle implementation are honest but potentially misconfigured at development time (e.g., due to programmer mistakes). The adversary can adaptively choose statements and messages and may observe multiple concurrent proofs.

Cryptographic assumptions: We assume the standard cryptographic assumptions underlying the interactive protocols and transforms: the Σ -protocols used as input are sound and (at least honest-verifier) zero-knowledge; the hash function realizing the random oracle is collision-resistant and behaves as an ideal random oracle for the purposes of the analysis; and the group or field operations used in commitments and responses are secure. YavOracle does not attempt to verify these assumptions or reason about algebraic vulnerabilities.

Implementation focus: Our goal is to rule out implementation-level coverage bugs in the realization of FS and Fischlin transforms. If the protocol specification itself omits a necessary binding (for example, if a published design erroneously fails to mention a field in the FS query), then YavOracle can still faithfully implement that flawed design. Conversely, if a value never appears in the symbolic transcript at

all because a constraint is missing from an underlying arithmetic circuit, this lies outside our scope; our framework operates at the transcript layer, not at the circuit synthesis layer.

Guarantees: Under these assumptions, coverage safety ensures that for any field explicitly marked as prover-controlled and verification-relevant, any attempt to use the YavOracle APIs in a way that fails to bind that field before deriving challenges or final decisions will be rejected at compile time. In particular, a Fiat–Shamir-style bug where a prover-controlled commitment or response element is omitted from the hash—as in the Solana ZK ElGamal case—cannot arise if that element is declared as obligatory: the type-level obligations and the `prove! / verify!` coverage checks prevent the verifier from being instantiated without an `absorb` call covering it.

Non-goals: We do not claim to prevent all classes of bugs in zero-knowledge systems. Logical errors in the verification predicate, misuse of secret randomness, incorrect group parameters, or missing arithmetic constraints are out of scope. Our contribution is to eliminate, as far as possible, the specific and pervasive class of FS/Fischlin omission bugs where a value that is understood to be part of the transcript in the design is inadvertently left out in the implementation.

The remainder of the paper develops a formal model and implementation strategy that realize coverage safety in Rust, with a particular emphasis on making the property transform-agnostic: the same coverage guarantees are enforced whether the runtime is interactive, Fiat–Shamir, or Fischlin with n-special soundness.

4 Design Overview: Coverage-Safe Transcript Programming

This section presents the high-level design of YavOracle. We describe how protocol authors specify interactive proofs using a typed message DSL, how coverage obligations are tracked as an effect on transcript types, and how a unified oracle abstraction exposes interactive, Fiat–Shamir, and Fischlin runtimes. We conclude with the role of the `prove! / verify!` macros, which provide transform-agnostic coverage checking at the proof level.

4.1 Design Objectives

The design is guided by four primary objectives:

- **Coverage safety:** The framework must enforce that every prover-controlled, verification-

relevant value declared by the programmer is bound into the transcript before any challenge or final decision that depends on it is derived. Omission bugs should be rejected at compile time whenever such values are correctly annotated

- **Transform-agnostic protocol code:** Protocol code (messages, rounds, and verification checks) should be expressed once, as it would be for an interactive proof. Switching between interactive, Fiat–Shamir, and Fischlin realizations must require changing only the oracle type or a small configuration parameter, not rewriting protocol logic
- **Structured domain separation and canonical encoding:** The framework should enforce a disciplined scheme for labels and encodings, so that all bound values are encoded canonically and domain separation is systematic rather than ad hoc
- **Ergonomic integration into Rust:** The approach should be usable by practitioners who are comfortable with Rust but not necessarily with formal verification. To this end, we rely on derive macros, const generics, and declarative macros, and avoid requiring users to write proofs or annotations beyond lightweight field attributes.

The remainder of this section describes how these objectives are realized by three core components: a message-level DSL (`fsr-bind-derive`), an effect-typed transcript (`fsr-core`), and a proof-level DSL (`fsr-proof-dsl`) layered on top of a unified oracle abstraction.

4.2 Typed Message DSL and Obligation Annotation

At the boundary between protocol design and implementation, YavOracle asks developers to model each protocol message as a Rust struct whose fields correspond to transmitted or local values. The `fsr-bind-derive` crate provides a derive macro `#[derive(FsrBindable)]` which turns such structs into bindable messages by generating (i) a constant obligation mask and (ii) canonical binding code.

Concretely, developers annotate fields as follows:

`#[bind(ob = N)]` marks a field as verification-relevant and pre-challenge. This field is treated as prover-controlled state that must be absorbed into the transcript before any subsequent challenge.

Each such annotation contributes a bit $1 \ll N$ to the struct’s obligation mask.

`#[bind(skip)]` explicitly declares that a field is not part of the transcript (e.g., scratch space or logging). This annotation prevents accidental omission by forcing the programmer to make non-binding explicit.

`#[bind(label = "custom.label")]` and `#[bind(prefix = "...")]` control the domain-separation label under which the field is absorbed. By default, the label is `TypeName.field_name`, optionally preceded by a prefix.

The derive macro then emits an implementation of a `Bindable` trait of the form

```
const OBLIG_MASK: u128 — the bitwise OR of all
1 << N for fields annotated with #[bind(ob = N)]; and
fn bind(&self, ...) — a canonical binding routine
that calls the transcript’s absorb operation on
each covered field using the chosen labels and a fixed
CanonicalEncode codec for each type.
```

At the level of protocol description, this is the only place where developers identify which fields are security-critical for coverage. The rest of the framework treats `OBLIG_MASK` as the authoritative set of obligations associated with that message type.

For multi-message rounds, a `declare_round!` macro in `fsr-core` aggregates the obligation masks of all pre-challenge messages in that round into a single const bitmask. For example, a declaration such as

```
1 declare_round!(R1 = [MsgA, MsgB]);
```

expands to a type alias `type R1<O> = Transcript<{ MsgA::OBLIG_MASK | MsgB::OBLIG_MASK }, O>`, thereby defining the initial obligation set for the round in terms of the message types. This provides a bridge from field-level annotations to round-level coverage obligations.

4.3 Effect-Typed Transcripts and Obligation Discharge

The central abstraction in `fsr-core` is the transcript type

```
Transcript<PENDING, O>
```

where `PENDING` is a const-generic integer encoding the current set of outstanding coverage obligations as a bitmask, and `O` is an oracle type parameter instantiating the underlying randomness source.

The transcript supports two key categories of operations:

Absorption operations: Given a bindable message `M` with obligation mask `M::OBLIG_MASK`, and a

transcript `Transcript<P, 0>` whose pending mask `P` includes `M::OBLIG_MASK` as a subset, the method

```
1 fn absorb<const MASK: u128, M: Bindable
  >(self, label: &str, m: &M) ->
  Transcript<{ P & !MASK }, 0>;
```

returns a new transcript in which the obligations corresponding to `MASK` have been cleared. In typical usage, one calls `absorb::< M::OBLIG_MASK , _>(...)`, and the compiler checks that the const expression `P & !M::OBLIG_MASK` is well-formed: at the type level, the pending set monotonically decreases as obligations are discharged.

The `bind()` method generated by `\inlif FsrBindable` is called inside `absorb` to perform the actual `Absorb` invocations on the oracle with appropriate labels and encodings.

Challenge operations: Challenges are sampled via a method of the form

```
1 fn challenge<T: FromChallenge>(self,
  label: &str) -> (T, Transcript<0, 0>)
  ;
```

which is only implemented when `PENDING == 0` at the type level. That is, there is no `challenge` implementation for `Transcript<P, 0>` when `P != 0`; the call is simply ill-typed. This restriction is enforced using Rust’s `generic_const_exprs` to gate trait implementations and inherent methods on const predicates.

The effect of `challenge` is to sample a new challenge element `T` from the oracle, absorbing the given label and the current transcript state into the oracle’s internal hash, and to return a new transcript with pending mask `0` for that round.

This typestate discipline realizes the effect intuition from Section 4: pending obligations behave like a linear effect that accumulates at the start of a round and must be entirely discharged via absorption before a challenge is accessible. Because `PENDING` is part of the type, any protocol code that attempts to skip a binding step and proceed directly to `challenge()` will fail to type-check. This guarantees that, for all fields annotated as verification-relevant via `#[bind(ob = N)]`, the implementation cannot derive challenges before binding them.

Multi-round protocols are expressed as sequences of such transcripts, with each round type `Rk<0> = Transcript<Pending_k, 0>` describing a boundary between commitment and challenge. Post-challenge messages (e.g., responses) may or may not be bound, depending on the transform; in our design they are handled uniformly by the same `Bindable` and oracle facilities, but they do not contribute to the pre-challenge obligation set.

4.4 Oracle Abstraction and Swappable Runtimes

The framework abstracts over the source of randomness and challenge generation via an oracle trait, denoted informally as `Oracle / TranscriptRuntime`. The oracle exposes two operations:

- **absorb(label, bytes):** update the oracle’s internal state with labeled data
- **challenge<T>(label):** derive a new challenge element `T` from the current state and a label

The transcript type `Transcript<PENDING, 0>` is parameterized by an oracle type `O` implementing this trait. Different choices of `O` correspond to different runtimes:

- **Interactive runtime:** In the interactive setting, the prover and verifier each instantiate a transcript with dual oracle types: `InteractiveProverOracle`, which sends challenge requests to the verifier over a channel and receives challenge values, and `InteractiveVerifierOracle`, which samples challenge randomness from a PRG or RNG and sends it back over the channel.

Both oracles implement the same `absorb / challenge` interface, but the challenges are realized as explicit network messages between parties. The application’s message flow (the Σ -protocol messages themselves) is orthogonal to this; the interactive oracle only carries verifier coins.

- **Fiat–Shamir runtime:** In FS mode, `HashOracle` implements a programmable random oracle on top of a collision-resistant hash or sponge. Calling `absorb` updates its internal hash state with `(label, bytes)`, while `challenge<T>` hashes the current state with the given label and deterministically reduces the output to a suitable field or scalar type.

The same protocol code can thus be compiled in non-interactive form simply by instantiating `Transcript<PENDING, HashOracle>` instead of `Transcript<PENDING, InteractiveProverOracle>` on the prover’s side.

- **Fischlin runtime:** In Fischlin mode, a `FischlinOracle` maintains, in addition to a hash state, the book-keeping necessary to enforce the hash predicate over multiple repetitions and to implement straight-line extraction. Parameters such as the number of repetitions ρ , the predicate bit-length (b) , and the n -special

soundness parameter n_{special} are supplied via a `FischlinParams` structure, and the runtime enforces the bit-budget inequality discussed in Section 3.3.

Crucially, `FischlinOracle` presents the same absorb/challenge interface as `HashOracle` from the point of view of protocol code; the additional logic is internal to the oracle.

Since all three runtimes implement the same oracle interface, the core protocol implementation (message definitions, round declarations, and verification logic) remains unchanged. To swap runtimes, the programmer either changes a type alias (e.g., `type O = HashOracle; to type O = FischlinOracle<Params>;`) or selects `transform = "fs"` vs `transform = "fischlin"` in the proof DSL (see below). The coverage guarantees provided by the transcript tpestate are identical across runtimes: obligations must still be discharged before `challenge()` is available.

4.5 Proof-Level DSL and Transform-Agnostic Coverage

While the typed transcript API suffices for low-level protocols, many applications benefit from a higher-level description of proof emission and verification. The `fsr-proof-dsl` crate provides such a description via macros `prove!`, `verify!`, `verify_source!`, and `proof!`. Of particular relevance to coverage safety are `prove!` and `verify!`.

At a high level, these macros are parameterized as follows:

- A `transform` selector, `transform = "fs" | "fischlin"` which chooses between a `HashOracle`-backed FS runtime and a `FischlinOracle`-backed runtime
- A `require` list, `require = ["c_0", "c_1", ...]`, which enumerates coverage obligations at the level of labels. Each label in this list denotes a logical binding site (e.g., a particular commitment or component of a multi-proof) that must be absorbed before the first challenge in the proof or, in Fischlin mode, before finalizing the joint hash used in the predicate

- A binding closure:

```
1 bind = |o: &mut _, i: usize,
      msg_bytes: &[u8]| {
2 // calls to fsr_core::
  TranscriptRuntime::absorb(o, "
  c_0", ...)
```

```
3 // calls to fsr_core::
  TranscriptRuntime::absorb(o, "
  c_1", ...)
4 // ...
5 }
```

which describes how recorded message bytes are mapped to labeled absorption events

The macro expands into code that (i) uses the appropriate oracle type depending on `transform`, (ii) replays or constructs the transcript, and (iii) statically checks that, within the body of `bind`, there is at least one call to `Absorb` under each label listed in `require`. If the analysis fails—because a required label is never absorbed, or only absorbed under a different name—the compiler emits an error such as:

```
1 coverage: missing absorb() for labels
  before challenge: c_0. add: fsr_core
  ::TranscriptRuntime::absorb(o, "c_0",
  <bytes>)
```

This proof-level DSL thus enforces coverage safety at a granularity that complements the field-level obligations:

- At the type level, `Transcript<PENDING, O>` enforces that all fields annotated with `#[bind(ob = N)]` are bound before challenges; it does not know how those bindings are grouped into high-level proof components
- At the DSL level, `require = [...]` and the `bind` closure ensure that each logical component (e.g., each clause in a Σ -OR or Σ -AND composition) is bound under specified labels in both FS and Fischlin realizations.

The `proof!` macro builds on this foundation to generate complete proof modules (encoders, decoders, verifiers, and verifier source) whose coverage obligations are derived directly from the DSL specification, ensuring consistency between prover, verifier, and any exported verifier code.

In summary, the design of `YavOracle` realizes coverage-safe transcript programming by aligning three levels of abstraction:

1. **Message level:** annotations on struct fields identify verification-relevant, prover-controlled data and assign them obligation bits and labels
2. **Transcript level:** const-generic obligation masks in `Transcript<PENDING, O>` enforce that all such data are absorbed before challenges become available

3. **Proof level:** the `prove!/verify!` DSL ties labeled absorption events to FS and Fischlin run-times, checking that all required labels are covered regardless of the chosen transform

This layered design meets the objectives stated at the outset: coverage safety is enforced syntactically, protocol code is written once and reused across transforms, and the implementation burden on developers is limited to local annotations and macro invocations.

5 Formalization of Coverage Safety

This section gives a concise formalization of coverage safety and its relationship to our implementation model. We emphasize that the cryptographic security of the resulting schemes still rests on the underlying Fiat–Shamir and Fischlin transforms; our contribution is to ensure that the code-level realization respects the transcript structure assumed by those transforms.

5.1 Transcripts, Values, and Bindings

Fix a public statement x and an underlying interactive protocol between prover P and verifier V . An execution trace is a sequence of message exchanges and local computations: from this trace we extract an abstract transcript τ , which records only those events that are relevant to the FS or Fischlin transform.

We model τ as a finite sequence of events of two kinds:

$$\tau = (e_1, e_2, \dots, e_n)$$

with

$$e_i \in \text{absorb}(\ell, v), \text{challenge}(\ell, c),$$

where:

- ℓ ranges over a countable set of labels
- v ranges over a set $\mathcal{V}$ of values that may be derived from the prover’s state, the statement x , or verifier messages
- c ranges over challenge values drawn from some challenge space $\mathcal{C}$

An event $\text{absorb}(\ell, v)$ denotes that the implementation has updated its transcript state (e.g., the internal hash or sponge state) by incorporating the encoding of v under label ℓ . An event $\text{challenge}(\ell, c)$ denotes that the implementation has derived a new challenge value c from the current transcript state, under label ℓ . For the purposes of coverage, we do

not model verifier-local computations beyond these events.

Among the values in $\mathcal{V}$, we distinguish a subset $\mathcal{V}_P \subseteq \mathcal{V}$ of prover-controlled values: those that are chosen, or whose distribution is influenced, by the prover (e.g., commitments, responses, and prover-chosen auxiliary inputs). We write $\text{dep}(e)$ for the set of values in $\mathcal{V}$ on which a given event e may depend. For challenges, this includes all values that have been bound into the transcript so far, as well as any additional explicit inputs (e.g., statement x and labels).

At the level of protocol specifications, the standard FS and Fischlin proofs implicitly assume that for each prover-controlled value $v \in \mathcal{V}_P$ that can influence acceptance, the transcript contains at least one corresponding absorb event $\text{absorb}(\ell_v, v)$ prior to any challenge or acceptance decision that depends on (v) . Coverage safety makes this assumption explicit and enforces it syntactically in implementations.

5.2 Obligations and Coverage-Safe Implementations

We now formalize coverage safety using obligations. Let $\mathcal{O} = o_1, \dots, o_k$ be a finite set of obligations. Each obligation o_j is associated with:

- a prover-controlled value $v_j \in \mathcal{V}_P$
- a designated label ℓ_j under which v_j must be absorbed

For the purposes of this paper, the association (v_j, ℓ_j) is derived from the programmer’s annotations: each field marked `#[bind(ob = N)]` gives rise to an obligation o_N , and its binding code fixes ℓ_N . The set $\mathcal{O}$ thus represents the coverage obligations declared by the implementation.

A transcript state is a pair (S, σ) , where:

- $S \subseteq \mathcal{O}$ is the set of pending obligations
- σ is an abstract representation of the internal transcript state (e.g., a hash preimage), whose exact structure is irrelevant for coverage

We define a small-step relation on transcript states induced by events:

- For an absorb event $\text{absorb}(\ell, v)$, we set

$$(S, \sigma) \xrightarrow{\text{absorb}(\ell, v)} (S', \sigma'),$$

where σ' is the result of updating the internal state with (ℓ, v) , and

$$S' = S \setminus o_j \in S \mid v_j = v \wedge \ell_j = \ell.$$

That is, absorbing (ℓ, v) discharges any obligations whose designated value and label match (v, ℓ)

- For a challenge event $\text{challenge}(\ell, c)$, we set

$$(S, \sigma) \xrightarrow{\text{challenge}(\ell, c)} (S, \sigma'),$$

where σ' is the result of deriving a challenge under label ℓ from σ . Challenges do not, by themselves, alter the set S of pending obligations.

We write $(S_0, \sigma_0) \xrightarrow{\tau} (S_n, \sigma_n)$ for the reflexive, transitive closure over a transcript $\tau = (e_1, \dots, e_n)$.

Definition 1. Let $\mathcal{O}$ be a set of obligations as above and let τ be a transcript. We say that τ is coverage-safe with respect to $\mathcal{O}$ if, for every prefix $\tau' \cdot \text{challenge}(\ell, c)$ of τ , and for every initial state (S_0, σ_0) with $S_0 = \mathcal{O}$, we have

$$(S_0, \sigma_0) \xrightarrow{\tau'} (S', \sigma') \Rightarrow S' = \emptyset.$$

In other words, immediately before any challenge is derived, the pending obligation set must be empty.

This definition captures the intuition that all declared obligations must be discharged by absorb events before any challenge is sampled. Note that it is parametric in $\mathcal{O}$: if the specification or annotations fail to declare a necessary obligation, coverage safety with respect to $\mathcal{O}$ does not repair that error.

An implementation is coverage-safe if all transcripts it can produce in honest executions are coverage-safe in the sense of Definition 1, for the obligation set $\mathcal{O}$ induced by its field annotations. Our Rust typestate discipline enforces this property syntactically: Each obligation o_j is represented by a bit position j in a const-generic mask. The type of a transcript carries the current set of pending obligations as a mask `PENDING`. Absorption methods update the mask by clearing bits corresponding to discharged obligations, and the `challenge()` method is only provided for transcript types whose `PENDING` mask is the zero mask.

Under these typing rules, any well-typed program in the core API induces only coverage-safe transcripts, since it is impossible to construct a call to `challenge()` from a state with non-empty `S`. At the DSL level, the `require = [...]` parameter plays an analogous role, but at the level of labels and recorded events rather than individual fields.

5.3 Security Implications (Informal)

The above formalization is intentionally minimal; the cryptographic security of the overall system still relies on established theorems for the Fiat–Shamir and Fischlin transforms. Informally, we rely on the following reasoning:

Let Π be a Σ -protocol or public-coin proof system with a symbolic transcript semantics and let Π^{FS} and Π^{Fischlin} denote its Fiat–Shamir and Fischlin transforms in the Random Oracle Model, instantiated over a transcript that includes all prover-controlled, verification-relevant values in the required places.

Suppose that Π satisfies the usual conditions (e.g., special soundness, honest-verifier zero knowledge) under which Π^{FS} and Π^{Fischlin} are known to be sound and to provide proofs of knowledge in the ROM.

A YavOracle-based implementation of Π is coverage-faithful if there is a mapping from its declared obligations $\mathcal{O}$ and labels to the symbolic transcript variables in the proof of soundness, such that any coverage-safe transcript for $\mathcal{O}$ corresponds to a transcript in the symbolic model where all relevant variables are included in the random-oracle queries at the correct phases.

Under this correspondence, coverage safety ensures that the code-level transcripts seen by the hash oracle are a subset of the spec-level transcripts used in the security proof. In particular, omission bugs of the kind exemplified in Section 4.1 (where a prover-controlled value is omitted from the FS query) cannot occur in a coverage-faithful implementation, because such code would be ill-typed or rejected by the DSL coverage checks.

We do not attempt to re-prove the security of FS or Fischlin: rather, we ensure that the necessary precondition—correct inclusion of all declared prover-controlled, verification-relevant values in the relevant hash inputs—is enforced by the programming model. In this sense, YavOracle acts as a lightweight bridge between the symbolic transcript semantics assumed in cryptographic analyses and the concrete Rust code used in deployed systems.

6 Implementation in Rust: Yav-Oracle Framework

Here we describe how the design is realized in Rust. We focus on the crate structure, the core typestate machinery, the binding `derive` macro, and the proof-level DSL that covers both Fiat–Shamir and Fischlin modes.

6.1 Crate Architecture and Toolchain

The implementation is organized into three crates: `fsr-core` (core transcript and oracle abstractions, with low-level macros for round declarations and examples), `fsr-bind-derive` (a procedural macro crate that generates `Bindable` implementations and obligation masks from annotated message structs), and `fsr-proof-dsl` (a macro DSL for coverage-checked `prove!`, `verify!`, `verify_source!`, and end-to-end `proof!` modules).

The core crate relies on `generic_const_exprs` to encode obligation masks as `const` generics on transcript types, and therefore currently targets nightly Rust.

Examples in `fsr-core/examples/` exercise both the low-level API and the DSL: Schnorr in interactive and FS mode, Σ -OR and Σ -AND protocols with coverage-checked FS and Fischlin variants, and an end-to-end proof generator using the `proof!` macro.

6.2 Core Types and Traits in `fsr-core`

The central type in `fsr-core` is the parametric transcript

```
1 pub struct Transcript<const PENDING:
    u128, 0> {
2     oracle: 0,
3     /* */
4 }
```

where `PENDING` is a bitmask of outstanding coverage obligations and `0` is an oracle implementing the transcript runtime.

Two families of methods are exposed:

Absorption: For a bindable message `M` with `const OBLIG_MASK: u128`, the transcript exposes methods of the form

```
1 impl<const P: u128, 0> Transcript<P, 0>
2 {
3     pub fn absorb<const MASK: u128, M:
4         Bindable>(
5         self,
6         label: &str,
7         m: &M,
8     ) -> Transcript<{ P & !MASK }, 0> { /*
9         */ }
```

In typical use, `MASK` is `M::OBLIG_MASK`. The `const` expression in the return type ensures that obligations represented by `MASK` are removed from `PENDING` at the type level. The body delegates to `M::bind` to emit the sequence of `Absorb` calls into the underlying oracle.

Challenges: Challenges are gated by the invariant `PENDING == 0`. This is encoded by implementing

challenge only for the zero mask:

```
1 impl<0> Transcript<0, 0> {
2     pub fn challenge<T: FromChallenge>(
3         self,
4         label: &str,
5     ) -> (T, Transcript<0, 0>) { /* */
6 }
```

There is no corresponding `impl` for `Transcript<P, 0>` with `P != 0`. Any attempt to call `challenge` before all obligations are discharged is therefore rejected at compile time as a missing method.

Round boundaries are declared via `declare_round!`, which aggregates the `OBLIG_MASKs` of all pre-challenge messages in that round:

```
1 declare_round!(R1 = [Commit]);
2 /* expands to: type R1<0> = Transcript<{
3     Commit::OBLIG_MASK }, 0>; */
```

`fsr-core` defines several traits for encoding and binding:

- **CanonicalEncode:** canonical byte encoding for types absorbed into the transcript (group elements, scalars, etc.)
- **Absorb:** an interface implemented by oracles for absorbing labeled byte slices
- **Oracle / TranscriptRuntime:** an interface for transcript oracles, giving `Absorb` and `challenge` operations against an internal state
- **Message:** describes protocol messages with metadata such as direction and a default label
- **Bindable:** describes types that can bind themselves into the transcript and carry an obligation mask:

```
1 trait Bindable {
2     const OBLIG_MASK: u128;
3     fn bind<O: Absorb>(&self, oracle:
4         &mut O, label_prefix: &str);
5 }
```

The derive macro (Section 7.3) generates `Bindable` impls from annotations on message structs, and `declare_round!` uses `OBLIG_MASK` to seed the `PENDING` mask for each round.

6.3 Binding Derive Macro: `fsr-bind-derive`

The `fsr-bind-derive` crate provides `#[derive(FsrBindable)]`, which turns a Rust struct into a

Bindable message. It uses `syn` and `quote` to inspect field attributes and generate code that computes a static obligation mask, and emits canonical absorption logic for each covered field.

Field-level attributes include:

- `#[bind(ob = N)]` Marks the field as pre-challenge, verification-relevant, and assigns it obligation bit $1 \ll N$. All such bits are logically ORed into `OBLIG_MASK`
- `#[bind(skip)]` Marks the field as intentionally not transcript-bound (scratch, logging, etc.). This prevents silent omission: excluding a field without an explicit `skip` will generate a compile error in the macro
- `#[bind(label = "custom.label")]` Overrides the default label, which is otherwise `TypeName.field_name` with an optional struct prefix

Struct-level attributes include:

- `#[bind(prefix = "TypePrefix")]` sets a label prefix for all fields
- `[bind(core = "fsr_core")]` allows the macro to be used from other crates via a different path to the core crate

For example, the Schnorr commitment message is declared as:

```
1 #[derive(Clone, Debug, FsrBindable)]
2 #[bind(prefix = "Commit")]
3 pub struct Commit {
4     #[bind(ob = 0)]
5     pub T: G1,
6 }
```

and the macro generates (schematically):

```
1 impl Bindable for Commit {
2     const OBLIG_MASK: u128 = 1 << 0;
3
4     fn bind<O: Absorb>(&self, oracle: &mut O, prefix: &str) {
5         oracle.absorb("Commit.T", &self.T);
6     }
7 }
```

Together with `declare_round!`, these derived implementations connect programmer annotations directly to the typestate enforced by `Transcript<PENDING, O>`.

6.4 Oracle Implementations and Runtime Swapping

The oracle abstraction allows the same protocol code to run under interactive, FS, or Fischlin runtimes. Each oracle implements `Absorb` and the challenge-generation interface expected by `Transcript<PENDING, O>`

`HashOracle` is a non-interactive programmable random oracle implemented over a hash/sponge. `Absorb` updates the internal state with `(label, bytes)`, and `challenge<T>` hashes the current state and label and reduces the output to type `T` via `FromChallenge`.

As for interactive oracles under the interactive feature, `InteractiveProverOracle` and `InteractiveVerifierOracle` communicate via a duplex `Channel` (instantiated in examples by `mem_duplex_pair()`). The verifier samples challenges from a PRNG (e.g., `StdRng::from_entropy()`), while both sides share the same absorb/ challenge interface as `HashOracle`.

Regarding the Fischlin Oracle, `FischlinOracle` implements Fischlin's transform, parameterized by `FischlinParams` containing `(, b, _c, n_special)`. It maintains hash state plus internal counters to enforce the predicate budget inequality $(b - \lceil \log_2(n_{\text{special}} - 1) \rceil) \geq c$. If the inequality fails or `b` is too small, it returns an `UnsoundParams` error or rejects proofs.

Swapping runtimes requires only changing the oracle type:

```
1 // FS:
2 let tr: R1<HashOracle> = Transcript::new(
3     HashOracle::new(b"YavOracle/FS"));
4
5 // Interactive:
6 let (p_chan, v_chan) = mem_duplex_pair();
7
8 let tr_p: R1<_> = Transcript::new(
9     InteractiveProverOracle::new(p_chan,
10     b"YavOracle/Int"));
11 let tr_v: R1<_> = Transcript::new(
12     InteractiveVerifierOracle::new(v_chan,
13     rng, b"YavOracle/Int"));
14
15 // Fischlin:
16 let params = FischlinParams { /*
17     including n_special */ };
18 let tr_f: R1<_> = Transcript::new(
19     FischlinOracle::new(params, b"
20     YavOracle/Fischlin"));
```

The typestate-driven coverage guarantees are identical across these runtimes.

6.5 Proof-Level DSL: `fsr-proof-dsl`

To avoid hand-wiring the transcript for every protocol, the `fsr-proof-dsl` crate exposes macros that operate at the level of labeled absorption events. `prove!` and `verify!` take a common configuration:

- `transform = "fs" | "fischlin"` — selects `HashOracle` or `FischlinOracle`.
- `require = ["c_0", "c_1", ...]` — a list of labels that must be absorbed before the first challenge (FS) or before finalizing the common hash (Fischlin)
- `bind = |o: &mut _, i: usize, m_bytes: &[u8]| { ... }` — a closure that replays the transcript by calling `fsr_core::TranscriptRuntime::absorb(o, "label", bytes)` for each required label

The macro performs a syntactic analysis of the `bind` closure, identifies the labels passed to `Absorb`, and checks that all labels listed in `require` appear at least once. If a required label is missing, the expansion fails with a compile-time error that includes a concrete suggestion, e.g.:

```
1 coverage: missing absorb() for labels
  before challenge: c_0. add: fsr_core
  :: TranscriptRuntime::absorb(o, "c_0",
    <bytes>)
```

Examples under `fsr-core/examples/` explicitly contrast “OK” and “bug” versions for Σ -OR and Σ -AND protocols in both FS and Fischlin modes (e.g., `sigma_or_dsl_fs_ok.rs` vs. `sigma_or_dsl_fs_bug.rs`), demonstrating that omission bugs that would otherwise compile are rejected by the DSL

The higher-level `prove!` macro generates a complete module for a given proof schema:

- A `Proof` struct with `encode/decode`
- `prove(events: &[RecEvent]) -> Option<Proof>`, which uses `RecordingHashOracle` to scan a recorded transcript and emit a compact proof
- `verify(pub_in, &proof)` and `verify_bytes(pub_in, bytes)` functions
- `verifier_source() -> &'static str`, which returns the source of the generated verifier
- A `REQUIRED_LABELS` array capturing coverage obligations induced by the schema

An example (`sigma_and_proof_dsl_end_to_end.rs`) records a transcript for a Σ -AND protocol, generates a proof, verifies it, and prints the verifier source. Because the replay specification is single-sourced in the `proof!` block, the coverage discipline is enforced “by construction”: the same labeled absorptions drive proving, verification, and the generated verifier.

6.6 Examples and Testing

The repository includes several worked examples. `schnorr.rs` and `schnorr_interactive.rs` demonstrate runtime swapping while reusing the same message types and round declarations. FS and Fischlin Σ -OR / Σ -AND examples in both DSL and attribute-lint style (`sigma_or_fs_bug.rs`, `sigma_or_dsl_fischlin_bug.rs`, etc.) are designed to fail compilation if any coverage obligation is omitted. `sigma_and_dsl_fischlin_nspecial_ok.rs` configures `n_special = 3` and appropriate parameters (`p`, `b`, `k_c`) to test `n-special` soundness for the Fischlin transform. The runtime enforces the corresponding bit-budget inequality. These examples serve both as documentation and as regression tests: any refactoring that weakens the coverage checks or breaks the FS/Fischlin runtimes causes the examples either to fail to compile (for coverage regressions) or to fail at runtime verification.

7 Evaluation

We provide a brief empirical evaluation to illustrate two aspects of the framework: (i) interchangeability of Fiat–Shamir and Fischlin transforms, and (ii) compile-time detection of omission bugs via coverage checks. A full performance study is left to future work.

7.1 Transform Interchange and Proof Size

The `fs_fischlin_interchange` example exercises the same Σ -protocol under both FS and Fischlin runtimes using the `prove!/verify!` DSL with `transform = "fs"` and `transform = "fischlin"`. Running this example yields:

```
1 FS ok = true, Fischlin ok = true
2 FS proof bytes: 38
3 Fischlin proof bytes: 504
```

Thus, the same protocol description and coverage specification produce accepting proofs in both modes, confirming that the DSL cleanly abstracts over the choice of transform. As expected, the

Fischlin proofs are significantly larger than the FS proofs, reflecting the additional repetitions and hash-predicate structure required for straight-line extraction and n -special soundness. The framework does not introduce excessive additional proof-size overhead beyond what is inherent to the underlying transforms.

7.1.1 Detection of Omission Bugs

To assess coverage checking, we use the `sigma_or_dsl_fs_bug` example, which deliberately omits a required binding in the `bind` closure. The example corresponds to a Σ -OR protocol specified via the DSL with

```
1 require = ["c_0", "c_1"];
    but the bind closure only calls absorb under label
    "c_1", commenting out the call for "c_0". Compiling
    this example produces the following error:
2 error: coverage: missing absorb() for
    labels before challenge: c_0. \
3 add: fsr_core::TranscriptRuntime::absorb
    (o, "c_0", <bytes>)
4 --> examples/sigma_or_dsl_fs_bug.rs
    :45:16
5 |
6 |         bind = |o: &mut _, _i:
7 |         | use, m_bytes: &[u8]| {
8 |         | | -----^
9 |         | | // BUG: forgot to
10 |         | | absorb c_0
11 |         | |
12 |         | | let _t0b = &m_bytes
13 |         | | [0..8];
14 |         | | let t1b = &m_bytes
15 |         | | [8..16];
16 |         | | fsr_core::
17 |         | | TranscriptRuntime::absorb(o, "c_1",
18 |         | | t1b);
19 |         | | },
20 |         | | -----^
```

The macro precisely identifies the missing label (`c_0`) and suggests the corrective action—adding `absorb(o, "c_0", <bytes>)`. Because compilation fails, the buggy FS implementation cannot be deployed, and the omission is repaired before any runtime tests. A Solana-style bug in which a prover-controlled component is silently omitted from the Fiat–Shamir transcript cannot arise if that component is declared as required in the DSL or annotated as obligatory at the message level.

These examples demonstrate that YavOracle’s implementation realizes the intended guarantees: protocols can be executed interchangeably under FS and Fischlin with consistent coverage, and typical omission bugs are rejected at compile time with actionable diagnostics.

8 Related Work

8.1 Fiat–Shamir Transcript Libraries

A large body of engineering work has focused on making Fiat–Shamir (FS) transcripts easier and safer to program. Merlin is a STROBE-based transcript construction that automates FS by providing a labeled `commit/challenge` interface, domain separation, framing, and transcript-based RNG for multi-round public-coin arguments. Merlin explicitly documents common “Fiat–Shamir problems”—ambiguous encodings, missing domain separation, and ad-hoc challenge derivation—but ultimately leaves it to the programmer to decide what to commit [4] [3].

Sponge-based FS libraries take this one step further. Spongefish is a permutation-agnostic FS library that exposes a duplex sponge API, interoperable across permutations (e.g., Keccak, Poseidon) and codecs, and is intended as a generic FS layer for public-coin protocols [12] [5]. The CFRG FS draft similarly specifies a stateful hash object with `absorb/squeeze` semantics for compiling interactive proofs to non-interactive ones via a duplex sponge interface. These designs normalize transcript shape and domain separation, and they are directly complementary to our work: YavOracle can be instantiated on top of such oracles.

More recently, Decree (Trail of Bits) builds a small layer on top of Merlin to predeclare expected inputs and challenges at each protocol stage and then enforce that the implemented transcript supplies them—“disarming Fiat–Shamir footguns” by constraining how code interacts with the underlying transcript [11]. Decree is conceptually closest to our goal: it adds structural constraints to ensure that particular names appear in the transcript. However, it does not track prover-controlled data as an effect or connect these names to a type-level description of protocol messages; the coverage property remains at the level of API discipline rather than being enforced at the level of Rust types and `const` generics. Work such as `curdleproofs`’ Merlin-based transcript wrapper follows a similar pattern, tailoring a transcript API to a protocol family without exposing coverage guarantees.

YavOracle differs from all of these in that it treats “prover-controlled, verification-relevant” data as a typed obligation that must be discharged before `challenge()` exists at the type level. It can use Merlin- or sponge-based oracles as backends, but adds a layer of compile-time taint-style tracking that existing transcript libraries do not attempt.

8.2 Zero-Knowledge DSLs and Circuit Analyses

In parallel, there is extensive work on domain-specific languages (DSLs) and verification tools for arithmetic circuits in zero-knowledge systems. Circom provides a language and compiler that generate R1CS constraints and witness code from high-level circuit descriptions, enabling modular composition of templates and making it easier to test and audit circuits [1]. Halo2 offers a Rust-based arithmetization for zkSNARKs, with constraint systems expressed as polynomial relations over field variables; recent work builds static and lightweight formal analyses of Halo2 circuits to catch under-constrained or ill-typed gadgets [2] [24]. Coda introduces a statically typed language with refinement types and Coq-backed proofs to certify correctness properties of zero-knowledge circuits, revealing previously unknown vulnerabilities in Circom projects [21]. More recently, zkFuzz proposes a trace-constraint consistency test and a fuzzing framework for ZK circuits, systematically uncovering circuit-level bugs such as under-constraints [25].

These efforts address a distinct layer of the stack: they reason about constraint systems and the correspondence between computation and circuit, whereas YavOracle reasons about transcripts and the correspondence between protocol messages and FS/Fischlin hash inputs. Circuit DSLs can guarantee that a given arithmetic relation is enforced; they generally assume that the FS or polynomial-commitment layer correctly binds public inputs, commitments, and evaluations into the transcript. Our work is orthogonal: we assume the underlying Σ -protocols and circuits are sound, and ensure that their realization as FS/Fischlin-style non-interactive arguments does not omit prover-controlled, verification-relevant values from the transcript.

8.3 Verified Cryptographic Libraries and FS Footguns

Formal verification of cryptographic libraries has focused mainly on primitives and low-level protocol code. HACLS* and EverCrypt provide verified implementations of modern cryptographic algorithms (symmetric ciphers, MACs, hashes, curve operations) in F*/Low* with proofs of memory safety, functional correctness, and side-channel resistance, compiled to C and assembly [13] [23]. This line of work dramatically reduces bugs in the implementation of primitives, but it does not address how those primitives are orchestrated into FS transcripts or how prover-

controlled values are selected for hashing.

Systematization efforts have recently highlighted that many real-world SNARK and FS deployments are vulnerable not because primitives are faulty, but because the FS transformation is applied to a partial or misordered transcript. For example, the “Frozen Heart” and “last challenge” attacks exploit implementations where the final FS challenge is computed before all relevant messages are fixed, or where some public inputs are omitted from the transcript [10] [8] [9]. These analyses reinforce the need for tooling that addresses FS “footguns” at the API level, complementing primitive-level verification.

YavOracle sits between these two lines of work. It does not verify primitives or the algebraic soundness of protocols, but it provides a typed programming model that prevents a pervasive class of FS/Fischlin implementation bugs: omission of prover-controlled, verification-relevant values from the transcript. In that sense, it can be seen as a lightweight, specialization of the verification ethos of HACLS*/EverCrypt to the particular problem of transcript coverage, informed by recent case studies of FS failures and by the design philosophy of libraries like Merlin, spongefish, and Decree.

9 Discussion

9.1 Limitations

YavOracle targets a narrow but important slice of the correctness story: coverage of prover-controlled, verification-relevant values in Fiat-Shamir and Fischlin transcripts. Several limitations follow from this focus.

First, coverage safety is defined relative to the obligations declared by the programmer. If a protocol specification is itself flawed, or if a security-critical field is never marked as obligatory (or never appears as a field at all, e.g., because a constraint is missing in an underlying circuit), our framework will faithfully implement the flawed design. We prevent a particular class of implementation bugs; we do not certify the protocol.

Second, YavOracle’s guarantees are implemented via Rust typestate and procedural macros. This entails practical constraints: the current implementation relies on nightly features (`generic_const_exprs`), and the programming model is closely tied to Rust’s type system. Portability to other languages or verification tools would require a re-engineering of the effect-typed transcript abstraction.

Third, coverage safety only concerns what is bound into the transcript and when. It does not

address other common sources of vulnerabilities: incorrect verification predicates, misuse of randomness, side-channel leakage, weak parameter choices, or algebraic attacks on the underlying groups or hash functions. Finally, our DSL-level coverage analysis operates at the level of labels and syntactic calls to `absorb`; while it is deliberately conservative, we do not provide a mechanized proof that the macro analysis is complete with respect to arbitrary Rust control flow.

9.2 Applicability and Extensions

Despite these limitations, we believe the approach is broadly applicable within the zero-knowledge and proof-systems ecosystem. At the most straightforward level, YavOracle can be used as a drop-in FS/Fischlin layer for Σ -protocol-based subroutines inside larger proof systems or protocols: identification schemes, range proofs, commitment opening proofs, and components of SNARK-friendly constructions. The ability to swap interactive, Fiat-Shamir, and Fischlin runtimes without altering protocol code is particularly attractive for libraries that wish to support both interactive testing and non-interactive deployment.

Beyond direct adoption, the underlying abstractions suggest several extensions. The obligation-based typestate discipline could be generalized to multi-party or stateful protocols, where coverage must be preserved across sessions, epochs, or chain heights. The labeling and DSL machinery could be integrated with existing transcript libraries (e.g., sponge-based or Merlin-like backends) to provide coverage guarantees without changing hash primitives. On the verification side, one could pair YavOracle with circuit-level analyzers and formally verified primitives, yielding a stack in which constraints, transcripts, and low-level cryptography are all subject to complementary checks.

Finally, there is an opportunity to formalize the mapping between protocol specifications and annotations more rigorously—e.g., by generating obligations from a higher-level protocol description, or by proving that the DSL’s coverage conditions imply an appropriate correspondence with the symbolic models used in FS and Fischlin security proofs.

10 Conclusion and Future Work

This paper has argued that a significant fraction of real-world failures in Fiat-Shamir and Fischlin-style constructions arise not from exotic cryptanalytic breakthroughs, but from a mundane class

of implementation errors: omission of prover-controlled, verification-relevant values from the transcript. These “coverage bugs” invalidate the preconditions of the classical security theorems, yet they are not adequately addressed by existing transcript libraries, which focus on domain separation and transcript shape but leave the choice of what to hash entirely to the programmer.

We introduced coverage safety as an explicit design objective for implementations of FS and Fischlin transforms, and we presented YavOracle, a Rust framework that enforces this property by construction. At the message level, developers annotate fields as obligatory; a `derive` macro turns these annotations into `Bindable` implementations and congeneric obligation masks. At the transcript level, the `Transcript<PENDING, 0>` typestate tracks outstanding obligations as an effect: `absorption` discharges obligations, and `challenge()` is only available when the pending set is empty. At the proof level, the `prove !/verify!` DSL specifies required labels and a binding closure; the macros statically ensure that all required labels are absorbed, independently of whether the underlying runtime is an interactive oracle, a Fiat-Shamir hash oracle, or a Fischlin oracle with n -special soundness.

Case studies on Σ -protocols, Σ -OR/ Σ -AND compositions, and an FS/Fischlin interchange example demonstrate that the same protocol code can be executed under different transforms, with consistent coverage guarantees, and that representative omission bugs—including patterns analogous to those that affected deployed systems—are rejected at compile time with precise diagnostics. In this sense, YavOracle can be viewed as a lightweight bridge between the symbolic transcript semantics assumed in security proofs and the concrete Rust code used in implementations.

There are several natural directions for future work. On the theoretical side, one could formalize the correspondence between protocol specifications and annotations, and mechanize the claim that coverage-safe YavOracle programs realize the transcript conditions assumed in Fiat-Shamir and Fischlin proofs. On the engineering side, integrating YavOracle with mainstream transcript backends (e.g., standardized duplex sponges or Merlin-like constructions), and with circuit-level analyzers or verified primitive libraries, would yield a more comprehensive end-to-end assurance. Extending the obligation-based typestate discipline to multi-party and stateful protocols, including rollups and light clients, is another promising avenue. Finally, beyond Rust, exploring analogous designs in other strongly typed languages could fur-

ther disseminate the idea that coverage safety should be a first-class property of modern zero-knowledge infrastructure, rather than an informal discipline left to code review.

References

- [1] Circom 2 documentation. <https://docs.circom.io/>. Accessed: 2025-12-03.
- [2] The halo2 book. <https://zcash.github.io/halo2/>. Accessed: 2025-12-03.
- [3] Merlin: Composable proof transcripts for public-coin arguments of knowledge. <https://docs.rs/merlin/>. Rust crate documentation, accessed: 2025-12-03.
- [4] Merlin transcripts. <https://merlin.cool/>. Accessed: 2025-12-03.
- [5] spongefish: A duplex sponge Fiat-Shamir library. <https://github.com/arkworks-rs/spongefish>. Accessed: 2025-12-03.
- [6] Zero-knowledge proof. https://en.wikipedia.org/wiki/Zero-knowledge_proof. Accessed: 2025-12-03.
- [7] ZK bug tracker. <https://github.com/0xPARC/zk-bug-tracker>. Community-maintained collection of zero-knowledge bugs, accessed: 2025-12-03.
- [8] Coordinated disclosure of vulnerabilities affecting girault, bulletproofs, and plonk. <https://blog.trailofbits.com/2022/04/13/part-1-coordinated-disclosure-of-vulnerabilities-affecting-girault-bulletproofs-and-plonk/>, 2022. Trail of Bits blog series introducing the “Frozen Heart” vulnerability class.
- [9] The frozen heart vulnerability in girault’s proof of knowledge. <https://blog.trailofbits.com/2022/04/14/the-frozen-heart-vulnerability-in-giraults-proof-of-knowledge/>, 2022. Trail of Bits blog post, accessed: 2025-12-03.
- [10] Exploring the frozen heart vulnerability in the Fiat-Shamir scheme. <https://slowmist.medium.com/slowmist-exploring-the-frozen-heart-vulnerability-in-the-fiat-shamir-scheme-3ff179450624>, 2023. Accessed: 2025-12-03.
- [11] Disarming Fiat-Shamir footguns. <https://blog.trailofbits.com/2024/06/24/disarming-fiat-shamir-footguns/>, 2024. Trail of Bits blog post, accessed: 2025-12-03.
- [12] A Fiat-Shamir transformation from duplex sponges. [https://cronokirby.com/Refs/\(2025-03\)-A-Fiat%E2%80%93Shamir-Transformation-From-Duplex-Sponges](https://cronokirby.com/Refs/(2025-03)-A-Fiat%E2%80%93Shamir-Transformation-From-Duplex-Sponges), 2025. Preprint, accessed: 2025-12-03.
- [13] BHARGAVAN, K., BEURDOUCHE, B., ZINZINDOHOUE, J.-K., AND PROTZENKO, J. HACl*: A verified modern cryptographic library. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)* (2017).
- [14] CHEN, Y.-H., AND LINDELL, Y. Optimizing and implementing fischlin’s transform for uc-secure zero-knowledge. *Cryptology ePrint Archive* (2024).
- [15] DAO, Q., MILLER, J., WRIGHT, O., AND GRUBBS, P. Weak fiat-shamir attacks on modern proof systems. In *2023 IEEE Symposium on Security and Privacy (SP)* (2023), IEEE, pp. 199–216.
- [16] FIAT, A., AND SHAMIR, A. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO ’86* (1986), vol. 263 of *Lecture Notes in Computer Science*, Springer, pp. 186–194.
- [17] FISCHLIN, M. Communication-efficient non-interactive proofs of knowledge with online extractors. In *Advances in Cryptology – CRYPTO 2005* (2005), vol. 3621 of *Lecture Notes in Computer Science*, Springer, pp. 152–168.

- [18] FOUNDATION, S. Post mortem: Zk elgamal proof program bug, june 2025.
- [19] GONG, S. Uncovering the phantom challenge soundness bug in solana’s zk elgamal proof program, Jun 2025.
- [20] LITCHEV, Y. Yavoracle: Coverage-safe transcript programming for Fiat–Shamir and fischlin. <https://github.com/RovayL/YavOracle>, 2025. GitHub repository.
- [21] LIU, J., KRETZ, I., LIU, H., TAN, B., WANG, J., SUN, Y., PEARSON, L., MILTNER, A., DILLIG, I., AND FENG, Y. Certifying zero-knowledge circuits with refinement types. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), IEEE, pp. 1741–1759.
- [22] POINTCHEVAL, D., AND STERN, J. Security proofs for signature schemes. In *Advances in Cryptology – EUROCRYPT ’96* (1996), Springer.
- [23] PROTZENKO, J., ET AL. Evercrypt: A fast, verified, cross-platform cryptographic provider. <https://project-everest.github.io/assets/evercrypt.pdf>, 2019. Preprint.
- [24] SOURESHJANI, F. H., ET AL. Automated analysis of halo2 circuits. In *Proceedings of the Workshop on Formal Methods in the Field* (2023). Preprint available at <https://ceur-ws.org/Vol-3429/paper3.pdf>.
- [25] TAKAHASHI, H., KIM, J., JANA, S., AND YANG, J. zkfuzz: Foundation and framework for effective fuzzing of zero-knowledge circuits. <https://arxiv.org/abs/2504.11961>, 2025. arXiv preprint.